

MAKALAH
“TUGAS SISTEM OPRASI”

Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Oprasi

Dosen : Triawan Adi Cahyanto, M.Kom



Disusun Oleh:
Ragil Anggara Kasih Budiarto 2410651045

UNIVERSITAS MUHAMMADIYAH JEMBER
2025-2026

DAFTAR ISI

BAB I PROCESS MANAGER	3
BAB II FILE MONITOR.....	8
BAB III CUSTOM SHELL.....	12
BAB IV SYSTEM INFORMATION TOOL	16

PROCESS MANAGER

Buat program sederhana yang mengelola process menggunakan konsep forking dan inter process communication (IPC) dasar. Program ini harus mendemonstrasikan pemahaman tentang process creation, synchronization, dan komunikasi.

```
agilnggara@RagilAnggara: ~/ x + v
agilnggara@RagilAnggara:~$ mkdir tugas1
agilnggara@RagilAnggara:~$ cd tugas1
agilnggara@RagilAnggara:~/tugas1$ mkdir process_manager
agilnggara@RagilAnggara:~/tugas1$ cd process_manager
agilnggara@RagilAnggara:~/tugas1/process_manager$ nano process_manager.c
```

```
GNU nano 7.2 process_manager.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>

pid_t child[3];

double time_diff(struct timeval a, struct timeval b) {
    return (b.tv_sec - a.tv_sec) + (b.tv_usec - a.tv_usec) / 1e6;
}

int main() {
    printf("\n=== Tugas 1: Process Manager (NIM GANJIL - Shared Memory) ===\n");
    printf("[Parent] Program dimulai\n\n");

    int pipes[2][2];
    for (int i = 0; i < 2; i++) pipe(pipes[i]); // hanya 2 pipe (child 1 & 2)

    // Untuk shared memory untuk child 3
    key_t key = 1234;
    int shmid = shmget(key, 256, IPC_CREAT | 0666);
    if (shmid < 0) { perror("shmget"); exit(1); }

    struct timeval start, end;
    int status; char buf[512];

    // Untukk Child 2
    gettimeofday(&start, NULL);
    if ((child[0] = fork()) == 0) {
        close(pipes[0][0]);
        int n; long long fakt = 1;
        printf("[Child 1] Masukkan angka untuk faktorial: ");
        scanf("%d", &n);
        for (int i = 1; i <= n; i++) fakt *= i;
        sprintf(buf, "%lld", fakt);
        write(pipes[0][1], buf, strlen(buf) + 1);
    }

    ^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-] To Bracket
    ^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_/ Go To Line M-E Redo      M-6 Copy      ^Q Where Was
```

```
GNU nano 7.2 process_manager.c *
    exit(0);
}
waitpid(child[0], &status, 0);
gettimeofday(&end, NULL);
close(pipes[0][1]); read(pipes[0][0], buf, sizeof(buf));
printf("\n[Parent] Child 1 selesai (PID=%d) | Exit=%d | Time=%.4f detik\n",
    child[0], WEXITSTATUS(status), time_diff(start, end));
printf("[Parent] Hasil faktorial = %s\n", buf);

// Untuk Child 2
gettimeofday(&start, NULL);
if ((child[1] = fork()) == 0) {
    close(pipes[1][0]);
    int batas; char hasil[512] = "";
    printf("[Child 2] Masukkan batas bilangan prima: ");
    scanf("%d", &batas);
    for (int i = 2; i <= batas; i++) {
        int p = 1;
        for (int j = 2; j * j <= i; j++)
            if (i % j == 0) p = 0;
        if (p) sprintf(hasil + strlen(hasil), "%d ", i);
    }
    write(pipes[1][1], hasil, strlen(hasil) + 1);
    exit(0);
}
waitpid(child[1], &status, 0);
gettimeofday(&end, NULL);
close(pipes[1][1]); read(pipes[1][0], buf, sizeof(buf));
printf("[Parent] Child 2 selesai (PID=%d) | Exit=%d | Time=%.4f detik\n",
    child[1], WEXITSTATUS(status), time_diff(start, end));
printf("[Parent] Bilangan prima = %s\n\n", buf);

// Untuk Child 3 Sekaligus Shared Memory
gettimeofday(&start, NULL);
if ((child[2] = fork()) == 0) {
    char *shm = (char *)shmat(shmid, NULL, 0);
    char str[128];
    printf("[Child 3] Masukkan string: ");
    scanf("%s", str);
    int len = strlen(str);
    for (int i = 0; i < len / 2; i++) {
        char tmp = str[i]; str[i] = str[len - 1 - i]; str[len - 1 - i] = tmp;
    }
    strcpy(shm, str); // ini untuk kirim hasil ke shared memory
    shmdt(shm);
    exit(0);
}
waitpid(child[2], &status, 0);
gettimeofday(&end, NULL);

// Untuk Parent membaca dari shared memory
char *shm = (char *)shmat(shmid, NULL, 0);
printf("[Parent] Child 3 selesai (PID=%d) | Exit=%d | Time=%.4f detik\n",
    child[2], WEXITSTATUS(status), time_diff(start, end));
printf("[Parent] String terbalik = %s\n", shm);
shmdt(shm);
shmctl(shmid, IPC_RMID, NULL); // Untuk Menghapus Shared Memory

printf("\n=== Semua Proses Telah Diselesaikan ===\n");
return 0;
}
```

```
    for (int i = 0; i < len / 2; i++) {
        char tmp = str[i]; str[i] = str[len - 1 - i]; str[len - 1 - i] = tmp;
    }
    strcpy(shm, str); // ini untuk kirim hasil ke shared memory
    shmdt(shm);
    exit(0);
}
waitpid(child[2], &status, 0);
gettimeofday(&end, NULL);

// Untuk Parent membaca dari shared memory
char *shm = (char *)shmat(shmid, NULL, 0);
printf("[Parent] Child 3 selesai (PID=%d) | Exit=%d | Time=%.4f detik\n",
    child[2], WEXITSTATUS(status), time_diff(start, end));
printf("[Parent] String terbalik = %s\n", shm);
shmdt(shm);
shmctl(shmid, IPC_RMID, NULL); // Untuk Menghapus Shared Memory

printf("\n=== Semua Proses Telah Diselesaikan ===\n");
return 0;
}
```

Pengertian Code:

- ❖ Proses parent harus menggunakan waitpid() untuk menunggu proses childnya selesai terlebih dahulu, akan tetapi saat parent ingin melanjutkan atau berhenti/keluar, parent ini perlu mengumpulkan status child berhenti agar mencegah child menjadi zombie proses.
- ❖ WIFEXITED ini digunakan untuk memeriksa proses child apakah proses child itu diselesaikan dengan normal. Disaat parent sedang menunggu proses child dengan code wait() ataupun waitpid().
- ❖ WEXITSTATUS ini digunakan untuk mengambil nilai dari suatu proses child yang telah diselesaikan dengan normal.

- ❖ Gettimeofday() ini digunakan untuk mencatat suatu waktu mulai dan juga waktu berakhirnya sebuah proses child.
- ❖ Shared Memory ini berfungsi untuk komunikasi antara proses parent dan proses child secara langsung.
- ❖ Shmget() ini digunakan untuk membuat ataupun untuk mengambil Shared Memory, Shmget() ini biasanya digunakan untuk berkomunikasi antara proses Parent dan proses Child.
- ❖ Shmat() ini digunakan untuk menghubungkan sebuah Shared Memory ke suatu proses, agar proses Parent dan Proses Childnya bisa membaca ataupun menulis di suatu memory yang sama.
- ❖ Shmctl() ini digunakan untuk mengontrol suatu shared memory, contohnya untuk menghapus suatu shared memory.
- ❖ Variasi Unik NIM saya adalah Ganjil (5) berarti menggunakan Shared Memory, Shared memory ini dibuat oleh Shmget() dan di akses melalui Shmat(), setelah child itu menulis sebuah hasil ke shared memory, parent membaca sebuah hasil tersebut, dan kemudian hasil tersebut di hapus melalui Shmctl().

```
process_manager.c: In function 'main':
process_manager.c:94:18: error: stray '@' in program
   94 |     child[@], WEXITSTATUS(status), time_diff(start, end));
      |           ^
process_manager.c:94:19: error: expected expression before '[' token
   94 |     child[@], WEXITSTATUS(status), time_diff(start, end));
      |           ^
```

```
process_manager.c: In function 'main':
process_manager.c:99:58: error: expected ';' before ':' token
   99 |     printf("\n== Semua Proses Telah Diselesaikan ==\n");
      |                                     ^
```

Laporan;

- ❖ Zombie proses ialah suatu proses yang sudah selesai dilakukan, akan tetapi proses tersebut masih tersisa di tabel proses karena belum memanggil parentnya belum menggunakan /memanggil wait() ataupun waitpid().
Cara menghindari proses zombie;
 - Parent harus menggunakan wait() ataupun waitpid()
 - Double fork() metode ini membuarkan sebuah child parent baru untuk menghapus child yg telah selesai di jalankan.
- ❖ Error proses_manager 94 itu error dikarenakan huruf "@" tidak dikenali, dalam kode tersebut, seharusnya child[2] bukan child [@].
- ❖ Error proses_manager 99 itu error dikarenakan huruf/tanda baca ":" tidak dikenali di sebuah kode, seharusnya huruf/tanda baca tersebut menggunakan ";" bukan menggunakan ":"
- ❖ Jadi Solusi untuk kedua masalah itu sama yaitu;

```

agilnggara@RagilAnggara:~$ agilinggaraRagilAnggara:~/tugas1/process_manager$ strace ./process_manager
execve("./process_manager", ["/process_manager"], 0x7ffc55bcd400 /* 27 vars */) = 0
brk(NULL)                               = 0x5690c47ff000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x75f402fd0e00
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=20787, ...}) = 0
mmap(NULL, 20787, PROT_READ, MAP_PRIVATE, 3, 0) = 0x75f402fd8000
close(3)                                = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0\0\0\1\0\0\0\220\243\2\0\0\0\0\0..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0...., 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0\0...., 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x75f402c00000
mmap(0x75f402c28000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x75f402c28000
mmap(0x75f402db0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x75f402db0000
mmap(0x75f402dff000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x75f402dff000
mmap(0x75f402e05000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x75f402e05000
close(3)                                = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x75f402fd5000
arch_prctl(ARCH_SET_FS, 0x75f402fd5740) = 0
set_tid_address(0x75f402fd5a10)         = 1203
set_robust_list(0x75f402fd5a20, 24)     = 0
rseq(0x75f402fd6000, 0x20, 0, 0x53053053) = 0
mprotect(0x75f402dff000, 16384, PROT_READ) = 0
mprotect(0x56909fabf000, 4096, PROT_READ) = 0
mprotect(0x75f403016000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x75f402fd8000, 20787)           = 0
fstat(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0), ...}) = 0
getrandom("\x7b\x80\xfa\xcf\x72\xf7\x00\xcc", 8, GRND_NONBLOCK) = 8
brk(NULL)                               = 0x5690c47ff000
brk(0x5690c4820000)                     = 0x5690c4820000
write(1, "\n", 1)                       = 1
write(1, "=== Tugas 1: Process Manager (NI)..., 64=== Tugas 1: Process Manager (NIM GANJIL - Shared Memory) ===", 64)
write(1, "[Parent] Program dimulai\n", 25[Parent] Program dimulai
write(1, "\n", 1)                       = 1
pipe2([3, 4], 0)                        = 0
pipe2([5, 6], 0)                        = 0
shmget(0x4d2, 256, IPC_CREAT|0666)     = 4
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARPID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x75f402fd5a10) = 1204
wait4(1204, [Child 1] Masukkan angka untuk faktorial: 7
[!WIFEXITED(s) && WEXITSTATUS(s) == 0], 0, NULL) = 1204

```

```

--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=1204, si_uid=1000, si_status=0, si_etime=0, si_stime=0} ---
close(4) = 0
read(3, "5040\0", 512) = 5
write(1, "\n[Parent] Child 1 selesai (PID=1)..., 66
[Parent] Child 1 selesai (PID=1204) | Exit=0 | Time=3.5304 detik
) = 66
write(1, "[Parent] Hasil faktorial = 5040\n"..., 33[Parent] Hasil faktorial = 5040

) = 33
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x75f402fd5a10) = 1206
wait4(1206, [Child 2] Masukkan batas bilangan prima: 13
[!WIFEXITED(s) && WEXITSTATUS(s) == 0]), 0, NULL) = 1206
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=1206, si_uid=1000, si_status=0, si_etime=0, si_stime=0} ---
close(6) = 0
read(5, "2 3 5 7 11 13 \0", 512) = 15
write(1, "[Parent] Child 2 selesai (PID=12)..., 65[Parent] Child 2 selesai (PID=1206) | Exit=0 | Time=1.8213 detik
) = 65
write(1, "[Parent] Bilangan prima = 2 3 5 "..., 42[Parent] Bilangan prima = 2 3 5 7 11 13

) = 42
clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD, child_tidptr=0x75f402fd5a10) = 1207
wait4(1207, [Child 3] Masukkan string: Ragil
[!WIFEXITED(s) && WEXITSTATUS(s) == 0]), 0, NULL) = 1207
--- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=1207, si_uid=1000, si_status=0, si_etime=0, si_stime=0} ---
shmat(4, NULL, 0) = 0x75f402fdd000
write(1, "[Parent] Child 3 selesai (PID=12)..., 65[Parent] Child 3 selesai (PID=1207) | Exit=0 | Time=3.0233 detik
) = 65
write(1, "[Parent] String terbalik = ligaR"..., 33[Parent] String terbalik = ligaR
) = 33
shmdt(0x75f402fdd000) = 0
shmctl(4, IPC_RMID, NULL) = 0
write(1, "\n", 1
) = 1
write(1, "=== Semua Proses Telah Diselesai"..., 40=== Semua Proses Telah Diselesaikan ===
) = 40
exit_group(0) = ?
+++ exited with 0 +++
agilnggara@RagilAnggara:~/tugas1/process_manager$ |

```

FILE MONITOR

Buat program yang memantau perubahan pada direktori tertentu dan mencatat aktivitas file, menggunakan system calls untuk file operations.

```
agilnggara@RagilAnggara:~$ cd tugasfilemonitor
agilnggara@RagilAnggara:~/tugasfilemonitor$ cd file_monitor
agilnggara@RagilAnggara:~/tugasfilemonitor/file_monitor$ nano file_monitor.c
```

```
GNU nano 7.2 file_monitor.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/inotify.h>
#include <sys/types.h>
#include <fcntl.h>
#include <signal.h>
#include <time.h>
#include <errno.h>

#define EVENT_SIZE (sizeof(struct inotify_event))
#define EVENT_BUF_LEN (1024 * (EVENT_SIZE + 16))
#define LOG_FILE "file_activity.log"

int inotify_fd, watch_fd, log_fd;

void handle_signal(int sig) {
    printf("\nSinyal %d diterima. Melakukan pembersihan...\n", sig);
    if (watch_fd >= 0)
        inotify_rm_watch(inotify_fd, watch_fd);
    if (inotify_fd >= 0)
        close(inotify_fd);
    if (log_fd >= 0)
        close(log_fd);
    printf("Inotify file descriptor ditutup.\n");
    printf("Watch descriptor dihapus.\n");
    printf("Log file ditutup dengan bersih.\n");
    printf("Program pemantauan dihentikan dengan bersih.\n");
    exit(0);
}

void log_event(const char *event_type, const char *filename, pid_t pid) {
    time_t now = time(NULL);
    struct tm *t = localtime(&now);
    char timestamp[64];
    char log_entry[256];

    strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S", t);
    snprintf(log_entry, sizeof(log_entry),

        "[%s] [%s] %s [PID: %d]\n",
        timestamp, event_type, filename, pid);

    write(log_fd, log_entry, strlen(log_entry));
    printf("%s", log_entry);

    printf("\n [EMAIL NOTIF] File '%s' mengalami event %s oleh PID %d\n", filename, event_type, pid);
    printf("\n Mengirim email notifikasi (simulasi)\n\n");
}

int main() {
    char dir_path[256];
    char buffer[EVENT_BUF_LEN];
    int length, i = 0;

    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);

    printf("=== Pemantau File System Dimulai ===\n");
    printf("Masukkan direktori yang ingin anda pantau: ");
    scanf("%255s", dir_path);

    log_fd = open(LOG_FILE, O_WRONLY | O_CREAT | O_APPEND, 0644);
    if (log_fd < 0) {
        perror("Gagal membuka file log");
        exit(EXIT_FAILURE);
    }

    inotify_fd = inotify_init();
    if (inotify_fd < 0) {
        perror("inotify_init gagal");
        exit(EXIT_FAILURE);
    }

    watch_fd = inotify_add_watch(inotify_fd, dir_path,
                                IN_CREATE | IN_MODIFY | IN_DELETE);

    if (watch_fd < 0) {
        if (errno == ENOSPC)
            fprintf(stderr, "[ERROR] Batas maksimum inotify watch tercapai (ENOSPC).\n");
        else
            ;
    }
}
```



```

        perror("inotify_add_watch gagal");
        exit(EXIT_FAILURE);
    }

    printf("Direktori yang dipantau: %s\n", dir_path);
    printf("Log dicatat ke: %s\n", LOG_FILE);
    printf("Mode: NIM GANJIL → Simulasi email notifikasi aktif.\n");
    printf("Tekan Ctrl+C untuk menghentikan.\n\n");

    while (1) {
        i = 0;
        length = read(inotify_fd, buffer, EVENT_BUF_LEN);
        if (length < 0) {
            perror("read gagal");
            break;
        }

        while (i < length) {
            struct inotify_event *event = (struct inotify_event *)&buffer[i];
            if (event->len > 0) {
                // 🔥 Abaikan event dari file log agar tidak looping
                if (strcmp(event->name, LOG_FILE) == 0) {
                    i += EVENT_SIZE + event->len;
                    continue;
                }

                if (event->mask & IN_CREATE)
                    log_event("CREATED", event->name, getpid());
                else if (event->mask & IN_MODIFY)
                    log_event("MODIFIED", event->name, getpid());
                else if (event->mask & IN_DELETE)
                    log_event("DELETED", event->name, getpid());
            }
            i += EVENT_SIZE + event->len;
        }
    }

    handle_signal(0);
    return 0;
}

```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-J To Bracket
 ^X Exit ^R Read File ^L Replace ^U Paste ^J Justify ^_ Go To Line M-E Redo M-C Copy ^Q Where Was

Penjelasan Kode;

- ❖ inotify_init() ini berfungsi untuk menganalisis dan membuka koneksi ke inotify kernel linux, yang bertujuan suatu program tersebut memantau perubahan file secara langsung.
- ❖ inotify_add_watch() ini digunakan untuk mendaftarkan suatu file yang ingin di pantau oleh inotify, dan juga yang mendefinisikan detail dari suatu sistem file seperti ,pembuatan file,perubahan file,penghapusan file.
- ❖ Struct inotify_event ini ialah inti dari inotify, yang berfungsi sebagai wadah untuk menyimpan informasi detail tentang event pada file yang sedang di pantau oleh inotify, ini dikirim oleh kernel ke suatu program melalui read() yang berisi jenis event(mask) nama file yang berubah.
- ❖ Membaca Buffer ini dilakukan dengan metode perulangan
 - Panggil read() ini berfungsi untuk menyalin satu ataupun lebih peristiwa yang terjadi kedalam buffer.
 - Cek hasil , ini berfungsi jika nilai suatu read() lebih kecil dari 0 maka akan terjadi error, dan sebaliknya jika nilai suatu read() lebih dari 0 maka ada data event yang harus di proses,
 - Isi buffer definisikan atau deklarasikan variable contohnya adalah int i=0, ini digunakan untuk menelusuri suatu seluruh peristiwa, karena ini menggunakan metode perulangan maka variabelnya harus bertambah.
- ❖ log_event() ini berfungsi untuk menghasilkan timestamp dengan menggunakan time() dan juga localtime(). ini juga berfungsi untuk mencaatat ke file log melalui write().

- ❖ Signal Handling cara menggunakan ini adalah dengan menekan tombol Ctrl dan C secara bersamaan ,jika menekan itu akan memanggil handle_signal() untuk memberhentikan perulangan/menutup inotify dengan aman.

Laporan:

- ❖ File descriptor ialah integer yang mewakili resource I/O yang terdiri dari file ,socket,device yang dibuka melalui proses di linux,file descriptor ini berfungsi untuk baca_tulis.
- ❖ Poling program secara berulang ulang untuk memeriksa perubahan.
- ❖ Event-driven suatu program yang menunggu notifikasi dari kernel linux dan event driven ini hanya bertindak ssaat ada peristiwa.
- ❖ ENOSPC adalah kode error dari linux ,pada dasarnya ini sudah tidak ada penyimpanan yang tersisa,akan tetapi pada contest inotify ini bermaksud pada jumlah watch yang di buat sudah mencapai batas kapasitas sistem.

Cara menanganinya dengan:

- Tampilkan pesan error pada pengguna ,jangan memaksakan program untuk lanjut,karena pada dasarnya inotify tidak bisa memantau file baru.
- Tingkatkan batas jumlah watch melalui perintah terminal contohnya seperti root.
- ❖ Variasi Unik Nim ganjil adalah dengan menambahkan fitur email dummy,tetapi pesan ini hanya simulasi tidak emailnya tidak benar benar terkirim/diterima.

Pengujian:

Terminal 1

```
agilnggara@RagilAnggara:~/tugasfilemonitor/file_monitor$ nano file_monitor.c
agilnggara@RagilAnggara:~/tugasfilemonitor/file_monitor$ gcc -o file_monitor file_monitor.c
agilnggara@RagilAnggara:~/tugasfilemonitor/file_monitor$ ./file_monitor
=== Pemantau File System Dimulai ===
Masukkan direktori yang ingin anda pantau: /home/agilnggara/tugasfilemonitor/file_monitor
Direktori yang dipantau: /home/agilnggara/tugasfilemonitor/file_monitor
Log dicatat ke: file_activity.log
Mode: NIM GANJIL → Simulasi email notifikasi aktif.
Tekan Ctrl+C untuk menghentikan.
```

Terminal 2

```
agilnggara@RagilAnggara:~/tugasfilemonitor/file_monitor$ touch 2410651045.txt
agilnggara@RagilAnggara:~/tugasfilemonitor/file_monitor$ echo "hanya mencoba" > 2410651045.txt
agilnggara@RagilAnggara:~/tugasfilemonitor/file_monitor$ rm 2410651045.txt
```

Output di Terminal akan menjadi seperti ini

```
[2025-10-16 18:25:59] [CREATED] 2410651045.txt [PID: 602]
[EMAIL NOTIF] File '2410651045.txt' mengalami event CREATED oleh PID 602
→ Mengirim email notifikasi (simulasi)

[2025-10-16 18:26:25] [MODIFIED] 2410651045.txt [PID: 602]
[EMAIL NOTIF] File '2410651045.txt' mengalami event MODIFIED oleh PID 602
→ Mengirim email notifikasi (simulasi)

[2025-10-16 18:26:25] [MODIFIED] 2410651045.txt [PID: 602]
[EMAIL NOTIF] File '2410651045.txt' mengalami event MODIFIED oleh PID 602
→ Mengirim email notifikasi (simulasi)

[2025-10-16 18:26:39] [DELETED] 2410651045.txt [PID: 602]
[EMAIL NOTIF] File '2410651045.txt' mengalami event DELETED oleh PID 602
→ Mengirim email notifikasi (simulasi)

^C
Sinyal 2 diterima. Melakukan pembersihan...
Inotify file descriptor ditutup.
Watch descriptor dihapus.
Log file ditutup dengan bersih.
Program pemantauan dihentikan dengan bersih.
```

CUSTOM SHELL

Tingkatkan simple shell dengan fitur lanjutan, mendemonstrasikan parsing command dan process management.

```
agilnggara@RagilAnggara:~$ mkdir tugascustomshell
agilnggara@RagilAnggara:~$ cd tugascustomshell
-bash: cd: tugascustomshell: No such file or directory
agilnggara@RagilAnggara:~$ cd tugascustomshell
agilnggara@RagilAnggara:~/tugascustomshell$ mkdir custom_shell
agilnggara@RagilAnggara:~/tugascustomshell$ cd custom_shell
agilnggara@RagilAnggara:~/tugascustomshell/custom_shell$ nano custom_shell.c
```

```
GNU nano 7.2 custom_shell.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>

#define MAX 256
#define MAX_PIPE 10

// Fungsi parsing command berdasarkan delimiter
void split(char *input, char *delim, char **tokens) {
    char *tok = strtok(input, delim);
    int i = 0;
    while (tok != NULL) {
        tokens[i++] = tok;
        tok = strtok(NULL, delim);
    }
    tokens[i] = NULL;
}

// Eksekusi satu command (handle redirection & pipe)
void exec_command(char **args, int in_fd, int out_fd) {
    if (in_fd != 0) { dup2(in_fd, 0); close(in_fd); }
    if (out_fd != 1) { dup2(out_fd, 1); close(out_fd); }

    // Handle redirection sederhana
    for (int i = 0; args[i]; i++) {
        if (!strcmp(args[i], "<")) {
            int fd = open(args[i + 1], O_RDONLY);
            dup2(fd, 0); close(fd);
            args[i] = NULL;
        } else if (!strcmp(args[i], ">")) {
            int fd = open(args[i + 1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
            dup2(fd, 1); close(fd);
            args[i] = NULL;
        } else if (!strcmp(args[i], ">>")) {
            int fd = open(args[i + 1], O_WRONLY | O_CREAT | O_APPEND, 0644);
            dup2(fd, 1); close(fd);
            args[i] = NULL;
        }
    }
}
```

```

    }
}

execvp(args[0], args);
perror("execvp");
exit(1);
}

// Eksekusi pipeline (), dengan background opsional
void execute_pipeline(char *line, int background) {
    char *cmds[MAX_PIPE];
    int n = 0;

    // Pisahkan berdasarkan |
    split(line, "|", cmds);
    while (cmds[n]) n++;

    int in_fd = 0, fd[2];

    for (int i = 0; i < n; i++) {
        char *args[MAX];
        split(cmds[i], " \\t\\n", args);

        if (!args[0]) continue;

        // Built-in commands
        if (!strcmp(args[0], "cd")) { chdir(args[1] ? args[1] : getenv("HOME")); return; }
        if (!strcmp(args[0], "pwd")) { char buf[512]; getcwd(buf, sizeof(buf)); printf("%s\\n", buf); return; }
        if (!strcmp(args[0], "echo")) {
            for (int j = 1; args[j]; j++) printf("%s ", args[j]);
            printf("\\n"); return;
        }
        if (!strcmp(args[0], "exit")) exit(0);

        // Jika bukan built-in, buat proses baru
        if (i < n - 1) pipe(fd);

        pid_t pid = fork();
        if (pid == 0) {
            if (i < n - 1) dup2(fd[1], 1);

```

```

            if (pid == 0) {
                if (i < n - 1) dup2(fd[1], 1);
                if (i > 0) dup2(in_fd, 0);

                if (i < n - 1) { close(fd[0]); close(fd[1]); }
                if (in_fd != 0) close(in_fd);

                exec_command(args, 0, 1);
            } else {
                if (i > 0) close(in_fd);
                if (i < n - 1) {
                    close(fd[1]);
                    in_fd = fd[0];
                }
                if (!background) waitpid(pid, NULL, 0);
            }
        }
    }

    int main() {
        char line[MAX];

        while (1) {
            printf("Shell-2410651045$ ");
            fflush(stdout);
            if (!fgets(line, sizeof(line), stdin)) break;

            // Hapus newline
            line[strcspn(line, "\\n")] = 0;
            if (strlen(line) == 0) continue;

            // Deteksi background
            int background = 0;
            if (line[strlen(line) - 1] == '&') {
                background = 1;
                line[strlen(line) - 1] = 0;
            }

            execute_pipeline(line, background);
        }
    }
}

```

```

    }

    return 0;
}

```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location ^U Undo ^M-A Set Mark ^M-] To Bracket
 ^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line ^M-E Redo ^M-6 Copy ^Q Where Was

Penjelasan Kode;

- ❖ Fgets() ini biasanya berfungsi untuk membaca string input contohnya pada mouse dan keyboard dengan aman .
- ❖ Dup() ini digunakan untuk menduplikasi file descriptor akan tetapi ini tetap dalam kernel yang sama.

- ❖ Chdir() ialah salah satu sistem call dasar di linux yang digunakan sebagai management direktori contoh penggunaan nya pada saat memproses cd.
- ❖ Execpv() ini digunakan untuk menjalankan program baru, yang dimaksud disini ialah execpv ini tidak membuat proses baru akan tetapi ini mengganti proses yang sedang berjalan, dan data data dari program sebelumnya akan di hapus.
- ❖ strtok() ini untuk digunakan untuk memecah sebuah input menjadi beberapa bagian bagian perintah.
- ❖ Persaing Logic itu digunakan untuk memecah sebuah string menjadikannya beberapa perintah agar bias di jalankan di ecepcv().
- ❖ Pipe mengubah file descriptor
 - Pipe hanya menyediakan dua descriptor baru ,
 - Pipe tidak mengubah file descriptor secara langsung,

Jadi aliran data antara suatu proses akan terjadi seperti rantai, yang beartian ini saling berhubungan .

- ❖ Kode unik Nim ganjil kali ini adalah melakukan multiple pipeline , yang bermkasud untuk pipe bisa lebih dari satu dalam satu input.

Laporan;

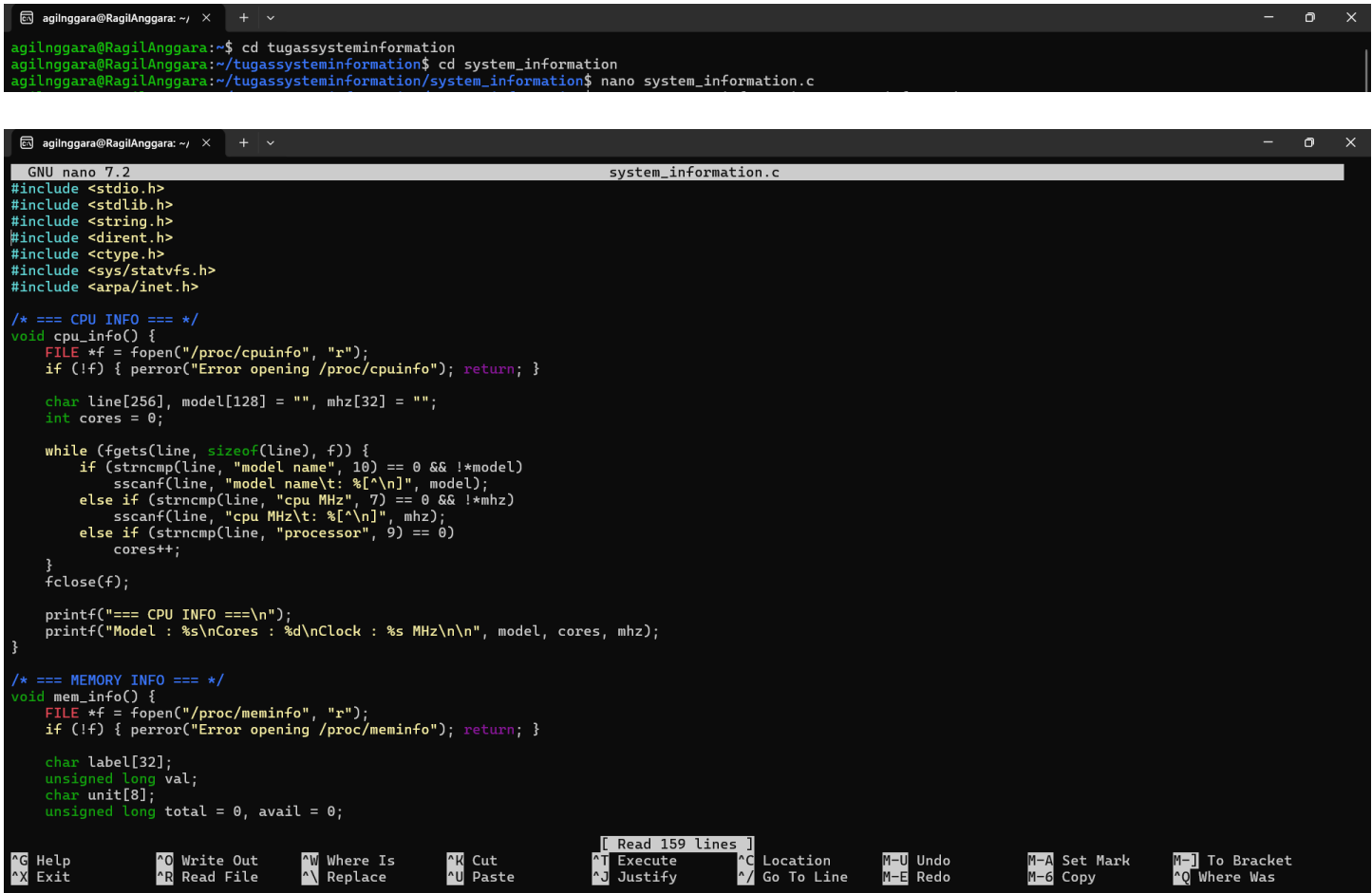
- ❖ Foreground Proses
 - Shell memanggil suatu fork () yang bermaksud untuk membuat suatu proses child,
 - Kemudian shell tersebut memanggil waitpid() yang bertujuan untuk menunggu proses child tersebut,
 - Terminal akan terkunci sementara sampai sampai semua proses child diselesaikan,
- ❖ Background Proses
 - Shell tetap memanggil suatu fork() akan tetapi di proses ini shell tidak memanggil wait ataupun waitpid(),
 - Shell juga tidak menunggu child menyelesaikan prosesnya,
 - Terminal juga masih tetap bisa digunakan untuk mengetik suatu perintah lain.

Pengujian;

```
Shell-2410651045$ ./custom_shell
agilnggara@RagilAnggara:~/tugascustomshell/custom_shell$ ./custom_shell
Shell-2410651045$ ls | grep txt > output.txt &
Shell-2410651045$ cat output.txt
contoh.txt
output.txt
Shell-2410651045$ ls | grep txt
contoh.txt
output.txt
Shell-2410651045$ exit
agilnggara@RagilAnggara:~/tugascustomshell/custom_shell$ gcc -g -o custom_shell custom_shell.c
agilnggara@RagilAnggara:~/tugascustomshell/custom_shell$ valgrind ./custom_shell
==674== Memcheck, a memory error detector
==674== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==674== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==674== Command: ./custom_shell
==674==
Shell-2410651045$ echo hanya mencoba
hanya mencoba
Shell-2410651045$ echo 2410651045 adalah nim saya
2410651045 adalah nim saya
Shell-2410651045$ ls | grep c
contoh.txt
custom_shell
custom_shell.c
Shell-2410651045$ ls | grep txt > hasil.txt
Shell-2410651045$ cat hasil.txt
contoh.txt
output.txt
Shell-2410651045$ exit
==674==
==674== HEAP SUMMARY:
==674==    in use at exit: 0 bytes in 0 blocks
==674==   total heap usage: 2 allocs, 2 frees, 2,048 bytes allocated
```

SYSTEM INFORMATION TOOL

Buat tool yang mengumpulkan dan menampilkan informasi sistem menggunakan system calls dan file proc.



The image shows a terminal window with a dark background. The top part shows the user navigating through directories and creating a new file. The bottom part shows the user editing a C program in nano. The program is designed to read system information from the /proc directory and print it to the console.

```
agilnggara@RagilAnggara: ~/
+
agilnggara@RagilAnggara:~$ cd tugassysteminformation
agilnggara@RagilAnggara:~/tugassysteminformation$ cd system_information
agilnggara@RagilAnggara:~/tugassysteminformation/system_information$ nano system_information.c
```

```
GNU nano 7.2 system_information.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <dirent.h>
#include <ctype.h>
#include <sys/statvfs.h>
#include <arpa/inet.h>

/* === CPU INFO === */
void cpu_info() {
    FILE *f = fopen("/proc/cpuinfo", "r");
    if (!f) { perror("Error opening /proc/cpuinfo"); return; }

    char line[256], model[128] = "", mhz[32] = "";
    int cores = 0;

    while (fgets(line, sizeof(line), f)) {
        if (strncmp(line, "model name", 10) == 0 && !*model)
            sscanf(line, "model name\t: %[\n]", model);
        else if (strncmp(line, "cpu MHz", 7) == 0 && !*mhz)
            sscanf(line, "cpu MHz\t: %[\n]", mhz);
        else if (strncmp(line, "processor", 9) == 0)
            cores++;
    }
    fclose(f);

    printf("=== CPU INFO ===\n");
    printf("Model : %s\nCores : %d\nClock : %s MHz\n\n", model, cores, mhz);
}

/* === MEMORY INFO === */
void mem_info() {
    FILE *f = fopen("/proc/meminfo", "r");
    if (!f) { perror("Error opening /proc/meminfo"); return; }

    char label[32];
    unsigned long val;
    char unit[8];
    unsigned long total = 0, avail = 0;

    [ Read 159 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   ^U Undo       ^M-A Set Mark ^M-] To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_/ Go To Line ^M-E Redo     ^M-6 Copy     ^Q Where Was
```



```
GNU nano 7.2 system_information.c

while (fscanf(f, "%31s %lu %7s\n", label, &val, unit) == 3) {
    if (!strcmp(label, "MemTotal:")) total = val;
    if (!strcmp(label, "MemAvailable:")) avail = val;
}
fclose(f);

printf("=== MEMORY INFO ===\n");
printf("Total : %.2f MB\nUsed : %.2f MB\n\n",
    total / 1024.0, (total - avail) / 1024.0);
}

/* === DISK INFO === */
void disk_info() {
    struct statvfs s;
    if (statvfs("/", &s) != 0) { perror("Error statvfs()"); return; }

    unsigned long long total = s.f_blocks * s.f_frsize;
    unsigned long long freeb = s.f_bavail * s.f_frsize;
    unsigned long long used = total - freeb;

    printf("=== DISK INFO ===\n");
    printf("Total : %.2f GB\nUsed : %.2f GB\n\n", total / 1e9, used / 1e9);
}

/* === PROCESS LIST === */
void list_process() {
    DIR *d = opendir("/proc");
    if (!d) { perror("Error opening /proc"); return; }

    struct dirent *e;
    printf("=== RUNNING PROCESSES ===\nPID\tNAME\n");

    while ((e = readdir(d))) {
        if (!isdigit(e->d_name[0])) continue;

        char path[256];
        char name[128];
        snprintf(path, sizeof(path) - 1, "/proc/%.200s/comm", e->d_name);
        path[sizeof(path) - 1] = '\0'; // pastikan null-terminated
    }
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-] To Bracket
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo      M-6 Copy      ^Q Where Was
```

```
GNU nano 7.2 system_information.c

    FILE *f = fopen(path, "r");
    if (f) {
        if (fgets(name, sizeof(name), f)) {
            name[strcspn(name, "\n")] = 0;
            printf("%s\t%s\n", e->d_name, name);
        }
        fclose(f);
    }
    closedir(d);
    printf("\n");
}

/* === NETWORK INTERFACE === */
void net_dev() {
    FILE *f = fopen("/proc/net/dev", "r");
    if (!f) { perror("Error opening /proc/net/dev"); return; }

    char line[256];
    fgets(line, sizeof(line), f);
    fgets(line, sizeof(line), f);

    printf("=== NETWORK INTERFACES ===\nIface\tRX(Bytes)\tTX(Bytes)\n");
    while (fgets(line, sizeof(line), f)) {
        char iface[32];
        unsigned long rx, tx;
        if (sscanf(line, "%31[^\n]: %lu %s %s %s %s %s %s %s %lu",
            iface, &rx, &tx) == 2)
            printf("%s\t%lu\t%lu\n", iface, rx, tx);
    }
    fclose(f);
    printf("\n");
}

/* === VARIASI NIM GANJIL: TCP CONNECTIONS === */
void net_tcp() {
    FILE *f = fopen("/proc/net/tcp", "r");
    if (!f) { perror("Error opening /proc/net/tcp"); return; }
}
```

```
GNU nano 7.2 system_information.c
char line[512];
fgets(line, sizeof(line), f); // skip header

printf("=== TCP CONNECTIONS (NIM GANJIL) ===\n");
printf("Local Address\tPort\tState\n");

while (fgets(line, sizeof(line), f)) {
    char local[64];
    int state;
    if (sscanf(line, "%d: %63s %s %x", local, &state) != 2) continue;

    char ip_hex[9] = {0}, port_hex[5] = {0};
    strncpy(ip_hex, local, 8);
    strncpy(port_hex, local + 9, 4);

    unsigned int ip_raw = strtoul(ip_hex, NULL, 16);
    unsigned char *b = (unsigned char *)&ip_raw;
    unsigned int port = strtoul(port_hex, NULL, 16);

    printf("%u.%u.%u.%u\t%u\t%x\n",
           b[0], b[1], b[2], b[3], port, state);
}
fclose(f);
printf("\n");
}

/* === MAIN === */
int main() {
    printf("=== SYSTEM INFORMATION TOOL (TUGAS 4) ===\n\n");

    cpu_info();
    mem_info();
    disk_info();
    list_process();
    net_dev();
    net_tcp();

    printf("=== END OF REPORT ===\n");
    return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute   ^G Location   M-U Undo      M-A Set Mark   M-J To Bracket
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify   ^/ Go To Line  M-E Redo      M-G Copy      ^Q Where Was
```

Penjelasan Kode;

- ❖ Struktur proc file sistem ialah sebuah representasi visual dari status kernel dan dari proses yang sedang berjalan, program ini membaca dengan menggunakan sistem call dasar contohnya seperti open, read, fscanf dan juga ini di parsing manual contohnya seperti fgets.
- ❖ statvfs() ini digunakan untuk mengecek berapa kapasitas total dan sisa ruang yang berada di sistem linux.
- ❖ Variasi Nim Ganjil [5] harus menambahkan fitur untuk menampilkan sebuah daftar koneksi, program membaca suatu file contohnya seperti /proc/net/tcp.

Laporan:

- ❖ Tujuan dari tugas ini adalah mengumpulkan dan menampilkan informasi sistem menggunakan system calls dan proc
- ❖ Struktur proc filesystem adalah salah satu fitur penting dan unik dari kernel, ini adalah filesystem virtual yang hanya ada di memori dan dibuat oleh kernel. Tujuan proc sendiri yaitu sebagai antarmuka konfigurasi dan status runtime.

Pengujian;

```
agilnggara@RagilAnggara: ~/ x + v
agilnggara@RagilAnggara:~/tugassysteminformation/system_information$ nano system_information.c
agilnggara@RagilAnggara:~/tugassysteminformation/system_information$ gcc -Wall -Wextra -o system_information system_information.c
agilnggara@RagilAnggara:~/tugassysteminformation/system_information$ ./system_information
===== SYSTEM INFORMATION TOOL (TUGAS 4) =====

=== CPU INFO ===
Model : AMD Ryzen 9 5900HX with Radeon Graphics
Cores : 16
Clock : 3293.737 MHz

=== MEMORY INFO ===
Total : 7638.18 MB
Used : 533.07 MB

=== DISK INFO ===
Total : 1081.10 GB
Used : 57.53 GB

=== RUNNING PROCESSES ===
PID    NAME
1      systemd
2      init-systemd(Ub
7      init
41     systemd-journal
91     systemd-udev
105    systemd-resolve
108    systemd-timesyn
176    cron
177    dbus-daemon
184    systemd-logind
186    wsl-pro-service
192    rsyslogd
199    agetty
208    agetty
209    unattended-upgr
295    SessionLeader
296    Relay(297)
297    bash
298    login
341    systemd
342    (sd-pam)
361    bash
455    system_informat

=== NETWORK INTERFACES ===
Iface  RX(Bytes)  TX(Bytes)

=== TCP CONNECTIONS (NIM GANJIL) ===
Local Address  Port  State
127.0.0.53     53    a
10.255.255.254 53    a
0.0.0.0 22    a
127.0.0.54     53    a

===== END OF REPORT =====
agilnggara@RagilAnggara:~/tugassysteminformation/system_information$ |
```

MAKALAH
“TUGAS 2 SISTEM OPRASI”

Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Oprasi

Dosen : Triawan Adi Cahyanto, M.Kom



Disusun Oleh:
Nama: Ragil Anggara Kasih Budiarto

NIM:2410651045

Prodi:Teknik Informatika

Kelas: TI - C

UNIVERSITAS MUHAMMADIYAH JEMBER

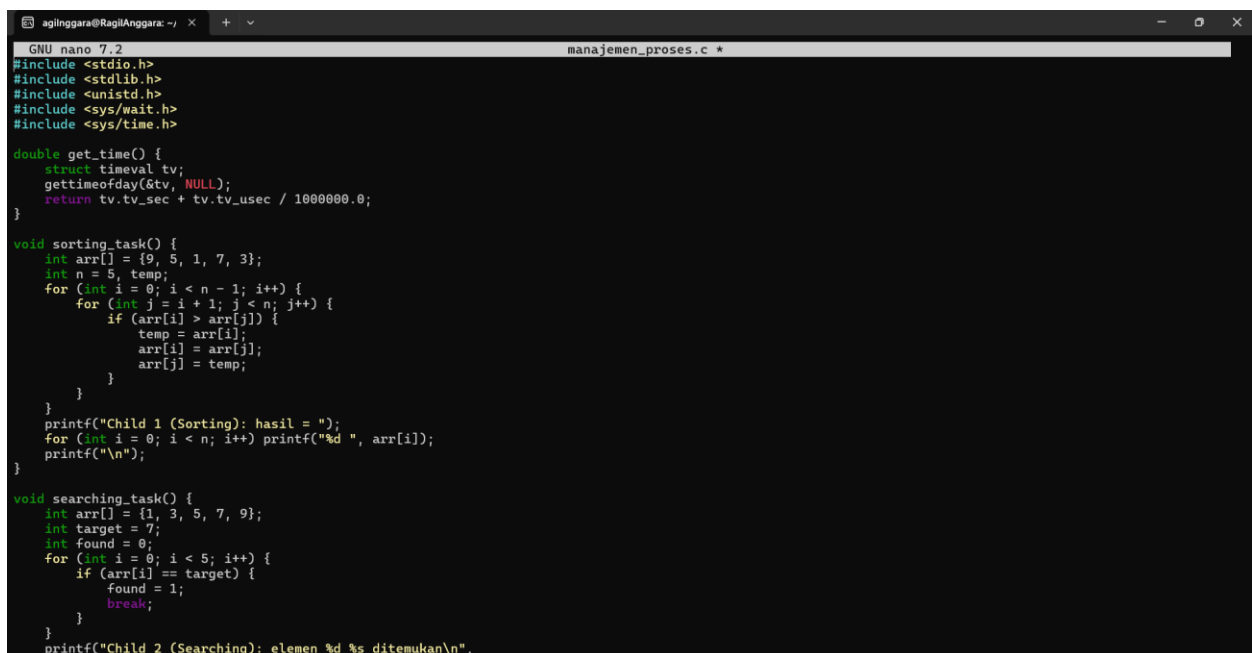
2025-2026

BAB I

SISTEM MANAJEMEN PROSES

Buat program C yang mensimulasikan sistem manajemen proses sederhana dengan fitur:
Membuat multiple child processes Setiap process melakukan tugas berbeda (sorting, searching, calculation) Parent process mengkoordinasi dan mengumpulkan hasil Tampilkan PID, execution time, dan status setiap proses.

Buat File manajemen_proses.c:



```
GNU nano 7.2                                manajemen_proses.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/time.h>

double get_time() {
    struct timeval tv;
    gettimeofday(&tv, NULL);
    return tv.tv_sec + tv.tv_usec / 1000000.0;
}

void sorting_task() {
    int arr[] = {9, 5, 1, 7, 3};
    int n = 5, temp;
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (arr[i] > arr[j]) {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }
    printf("Child 1 (Sorting): hasil = ");
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
}

void searching_task() {
    int arr[] = {1, 3, 5, 7, 9};
    int target = 7;
    int found = 0;
    for (int i = 0; i < 5; i++) {
        if (arr[i] == target) {
            found = 1;
            break;
        }
    }
    printf("Child 2 (Searching): elemen %d %s ditemukan\n",
```

```
target, found ? "" : "tidak");
}

void calculation_task() {
    long sum = 0;
    for (int i = 1; i <= 1000000; i++)
        sum += i;
    printf("Child 3 (Calculation): hasil penjumlahan = %ld\n", sum);
}

int main() {
    pid_t pid[3];
    double start, end;

    printf("=== Sistem Manajemen Proses ===\n");

    for (int i = 0; i < 3; i++) {
        start = get_time();
        pid[i] = fork();

        if (pid[i] == 0) {
            printf("\n[Child Process %d] PID = %d, Parent = %d\n",
                i + 1, getpid(), getppid());
            switch (i) {
                case 0: sorting_task(); break;
                case 1: searching_task(); break;
                case 2: calculation_task(); break;
            }
            end = get_time();
            printf("[Child %d] Execution time = %.6f detik\n",
                i + 1, end - start);
            exit(0);
        }
    }

    // Parent menunggu semua child selesai
    for (int i = 0; i < 3; i++) {
        int status;
        waitpid(pid[i], &status, 0);
        printf("[Parent] Child %d (PID = %d) selesai dengan status %d\n",
            i + 1, pid[i], WEXITSTATUS(status));
    }

    printf("\nSemua proses selesai!\n");
    return 0;
}
```

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set Mark M-J To Bracket
Exit Read File Replace U Paste J Justify / Go To Line M-E Redo M-G Copy Q Where Was

Compile dan Jalankan:

```
agilnggara@RagilAnggara:~/tugas2/manajemen_proses$ gcc manajemen_proses.c -o manajemen_proses
agilnggara@RagilAnggara:~/tugas2/manajemen_proses$ ./manajemen_proses

=== Sistem Manajemen Proses ===

[Child Process 1] PID = 713, Parent = 712
Child 1 (Sorting): hasil = 1 3 5 7 9
[Child 1] Execution time = 0.000784 detik

[Child Process 2] PID = 714, Parent = 712
Child 2 (Searching): elemen 7 ditemukan
[Child 2] Execution time = 0.001011 detik
[Parent] Child 1 (PID = 713) selesai dengan status 0
[Parent] Child 2 (PID = 714) selesai dengan status 0

[Child Process 3] PID = 715, Parent = 712
Child 3 (Calculation): hasil penjumlahan = 500000500000
[Child 3] Execution time = 0.002641 detik
[Parent] Child 3 (PID = 715) selesai dengan status 0

Semua proses selesai!
```

Penjelasan:

❖ Perisiapam

- Fungsi `get_time()` ini digunakan untuk mengukur sebuah waktu eksekusi `gettimeofday()`.
- Array PID ini disiapkan untuk menyimpan sebuah ID dari masing masing Child.

❖ Membuat Proses Child

Karena yang di tugaskan itu (Sorting,Search,dan Calculation) jadi ini hanya membuthhkan 3 Child saja,setiap masing masing child memiliki 1 proses tugas.

- Fork() digunakan dalam sebuah program loop 3 kali
- Setiap child menjalankan tugas yang berbeda:
 - Child 1 untuk menjalankan tugas sorting
 - Child 2 untuk menjalankan tugas searching
 - Child 3 unuk menjalankan tugas perhitungan

❖ Menjalankan Tugas

- Setiap child menghasilkan hasil kerjanya/output.
- Setelah selesai,child memcatat waktu eksekusi dan memanngil exit(0).

❖ Koordinasi Parent

- Parent menunggu setiap child menggunakan waitpid(),
- Setelah semua child menyelesaikan tugasnya ,parent menampilkan pid dan status exit tiap child.

❖ Akhir dari program

- Setelah seluruh child selesai menjalankan tugasnya,parent akan menampilkan pesan “Semua Proses Telah Diselesaikan”.

Analisis Hasil:

- ❖ Saat Forking berhasil ,ditunjukan olehn beberapa PID berbeda pada masing masing child.
- ❖ Eksekusi paralel (concurrent) terjadi ketika urutan hasil tidak selalu sama disaat program dijalankan.
- ❖ Parent tetap berjalan disaat semua child telah menyelesaikan tugasnya(Sinkornasinya berhasil dengan waitpid()).

Kesimpulan:

Program berhasil mensimulasikan sistem manajemen proses di mana setiap proses child menjalankan fungsi spesifik dengan waktu eksekusi berbeda, dan proses parent berperan sebagai pengendali utama yang menunggu dan mengelola seluruh proses child.

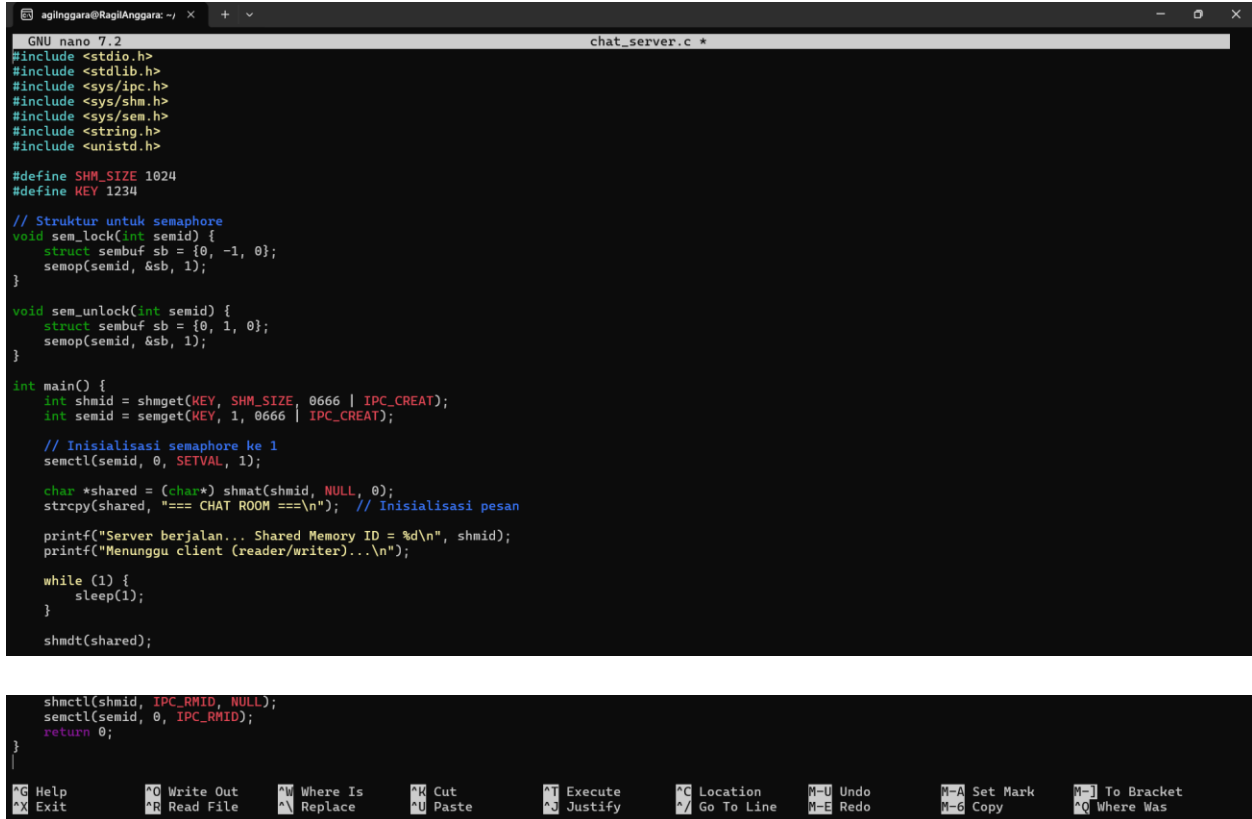
BAB II CHAT APPLICATION DENGAN IPC

Implementasikan aplikasi chat sederhana menggunakan:

- ❖ Shared Memory atau Message Queue untuk menyimpan pesan
- ❖ Multiple processes sebagai pengguna
- ❖ Mutex/Semaphore untuk sinkronisasi
- ❖ Fitur: kirim pesan, lihat history, exit

Buat file chat_server.c:

```
agilnggara@RagilAnggara:~$ cd ~/tugas2
agilnggara@RagilAnggara:~/tugas2$ mkdir chat_aplication
agilnggara@RagilAnggara:~/tugas2$ cd chat_aplication
agilnggara@RagilAnggara:~/tugas2/chat_aplication$ nano chat_server.c
```



```
GNU nano 7.2 chat_server.c *
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <string.h>
#include <unistd.h>

#define SHM_SIZE 1024
#define KEY 1234

// Struktur untuk semaphore
void sem_lock(int semid) {
    struct sembuf sb = {0, -1, 0};
    semop(semid, &sb, 1);
}

void sem_unlock(int semid) {
    struct sembuf sb = {0, 1, 0};
    semop(semid, &sb, 1);
}

int main() {
    int shmid = shmget(KEY, SHM_SIZE, 0666 | IPC_CREAT);
    int semid = semget(KEY, 1, 0666 | IPC_CREAT);

    // Inisialisasi semaphore ke 1
    semctl(semid, 0, SETVAL, 1);

    char *shared = (char*) shmat(shmid, NULL, 0);
    strcpy(shared, "=== CHAT ROOM ===\n"); // Inisialisasi pesan

    printf("Server berjalan... Shared Memory ID = %d\n", shmid);
    printf("Menunggu client (reader/writer)...\n");

    while (1) {
        sleep(1);
    }

    shmdt(shared);

    semctl(shmid, IPC_RMID, NULL);
    semctl(semid, 0, IPC_RMID);
    return 0;
}
```

Help Write Out Where Is Cut Execute Location M-U Undo M-A Set Mark M-] To Bracket
Exit Read File Replace Paste Justify Go To Line M-E Redo M-C Copy M-^ Where Was

Buat file chat_writer.c:


```
GNU nano 7.2 chat_writer.c *
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <string.h>
#include <unistd.h>

#define SHM_SIZE 1024
#define KEY 1234

void sem_lock(int semid) {
    struct sembuf sb = {0, -1, 0};
    semop(semid, &sb, 1);
}

void sem_unlock(int semid) {
    struct sembuf sb = {0, 1, 0};
    semop(semid, &sb, 1);
}

int main() {
    int shmid = shmget(KEY, SHM_SIZE, 0666);
    int semid = semget(KEY, 1, 0666);

    char *shared = (char*) shmat(shmid, NULL, 0);
    char message[100], username[20];

    printf("Masukkan nama user: ");
    fgets(username, sizeof(username), stdin);
    username[strcspn(username, "\n")] = 0; // hapus newline

    while (1) {
        printf("%s> ", username);
        fgets(message, sizeof(message), stdin);

        if (strncmp(message, "exit", 4) == 0) {
            printf("Keluar dari chat...\n");
            break;
        }

        sem_lock(semid);
        strcat(shared, username);
        strcat(shared, "> ");
        strcat(shared, message);
        sem_unlock(semid);
    }

    shmdt(shared);
    return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^R Cut        ^I Execute    ^C Location   ^U Undo       ^-A Set Mark  ^_] To Bracket
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_/ Go To Line ^=E Redo     ^-G Copy      ^_Q Where Was
```

Buat File chat_reader.c:

```
GNU nano 7.2 chat_reader.c *
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <string.h>
#include <unistd.h>

#define SHM_SIZE 1024
#define KEY 1234

void sem_lock(int semid) {
    struct sembuf sb = {0, -1, 0};
    semop(semid, &sb, 1);
}

void sem_unlock(int semid) {
    struct sembuf sb = {0, 1, 0};
    semop(semid, &sb, 1);
}

int main() {
    int shmid = shmget(KEY, SHM_SIZE, 0666);
    int semid = semget(KEY, 1, 0666);

    char *shared = (char*) shmat(shmid, NULL, 0);

    while (1) {
        sem_lock(semid);
        system("clear");
        printf("%s", shared);
        sem_unlock(semid);
        sleep(1);
    }

    shmdt(shared);
    return 0;
}
```

Compilasi dan Jalankan:

Terminal 1

```
agilnggara@RagilAnggara:~/tugas2/chat_aplication$ gcc chat_server.c -o chat_server
agilnggara@RagilAnggara:~/tugas2/chat_aplication$ ./chat_server
Server berjalan... Shared Memory ID = 0
Menunggu client (reader/writer)...
```

Terminal 2

```
agilnggara@RagilAnggara:~/tugas2/chat_aplication$ gcc chat_writer.c -o chat_writer
agilnggara@RagilAnggara:~/tugas2/chat_aplication$ ./chat_writer
Masukkan nama user: Nama saya ragil nim akhir 45
Nama saya ragil nim> Nama saya ragil nim> ^C
agilnggara@RagilAnggara:~/tugas2/chat_aplication$
```

Terminal 3

```
agilnggara@RagilAnggara:~/tugas2/chat_aplication$ ./chat_reader
```

```
=== CHAT ROOM ===
Nama saya ragil nim> akhir 45
```

Penjelasan:

- ❖ Program ini dibagi menjadi dua jenis proses:
 - Writer ini bertugas untuk mengirim pesan.
 - Reader ini bertugas untuk membaca pesan/melihat pesan.

- ❖ Kedua proses ini berbagi area memori(shared memory) untuk menyimpan teks percakapan.
- ❖ Ftok() ini berguna untuk membuat key unik di shared memory.
- ❖ Shmget() ini berfungsi sebagai membuat segmen shared memory berukuran 1042 byte.
- ❖ Writer Process
 - Menunggu giliran (lock semaphore)
 - Menulis pesan kepada shared memory
 - Unlock semaphore
- ❖ Reader Process
 - Mengambil pesan dari shared memory dan menampilkannya.

Analisi Hasil:

- ❖ Shared memory memungkinkan komunikasi cepat antar proses ,dikarenakan data tidak perlu dikirim melalui kernel buffer seperti FIFO.
- ❖ Program ini dapat menunjukkan bagaimana multi-user chat dapat disimulasikan tanpa jaringan.

Kesimpulan:

Aplikasi chat berbasis Shared Memory + Semaphore dapat berfungsi sebagai contoh nyata komunikasi antar proses (IPC).

Program ini memperlihatkan bagaimana sinkronisasi dan koordinasi antar proses bisa dilakukan tanpa menggunakan jaringan eksternal, hanya dengan memori bersama.